

Déployer, Démarrer et Exposer un Container avec Kubernetes



Guide pas à pas — commandes kubectl et explications

0. Créer l'image Docker (Dockerfile)

Avant de déployer sur Kubernetes, il faut construire l'image Docker de votre application et la publier sur un registre (Docker Hub, GitHub Container Registry, etc.) afin que le cluster puisse la télécharger.

Exemple de Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Construire l'image

```
docker build -t mon-compte/mon-app:1.0 .
```

Tester l'image en local

```
docker run -p 3000:3000 mon-compte/mon-app:1.0
```

Vérifiez sur <http://localhost:3000> que l'application répond correctement avant de continuer.

Se connecter au registre et publier l'image

```
docker login
docker push mon-compte/mon-app:1.0
```

Bon à savoir

Le nom d'image utilisé dans le Deployment (étape 1) doit correspondre exactement à celui poussé sur le registre : mon-compte/mon-app:1.0.

Évitez le tag 'latest' en production : un tag versionné (1.0, 1.1...) garantit que Kubernetes déploie la version attendue.

0.1. Créer un cluster local (minikube ou kind)

Pour tester ce guide sans cluster cloud, vous pouvez créer un cluster Kubernetes local sur votre machine avec minikube ou kind (Kubernetes IN Docker). Les deux s'appuient sur Docker et sont interchangeables — choisissez l'un ou l'autre.

Option A — minikube

minikube crée une VM (ou un conteneur) qui fait tourner un cluster Kubernetes complet.

```
minikube start --driver=docker
```

Vérifier que le cluster est prêt :

```
minikube status  
kubectl get nodes
```

Charger une image Docker construite en local directement dans le cluster (sans passer par un registre) :

```
minikube image load mon-compte/mon-app:1.0
```

Ouvrir le tableau de bord graphique :

```
minikube dashboard
```

Exposer un Service de type NodePort facilement (ouvre le navigateur) :

```
minikube service mon-app
```

Arrêter ou supprimer le cluster :

```
minikube stop  
minikube delete
```

Option B — kind (Kubernetes IN Docker)

kind fait tourner chaque node du cluster comme un simple conteneur Docker — très léger et rapide, idéal pour la CI.

```
kind create cluster --name mon-cluster
```

Vérifier que le cluster est prêt :

```
kubectl cluster-info --context kind-mon-cluster  
kubectl get nodes
```

Charger une image Docker construite en local directement dans le cluster :

```
kind load docker-image mon-compte/mon-app:1.0 --name mon-cluster
```

Lister les clusters kind existants :

```
kind get clusters
```

Supprimer le cluster :

```
kind delete cluster --name mon-cluster
```

💡 Bon à savoir

minikube vs kind : minikube offre plus d'outils intégrés (dashboard, addons, tunnel...) et convient bien au poste de travail. kind est plus léger et rapide, très utilisé en intégration continue (CI).

Avec les deux, l'image chargée via 'image load' ou 'load docker-image' n'a pas besoin d'être poussée sur un registre distant — pratique pour itérer rapidement en local.

0.2. Faut-il créer des dossiers ?

La création de dossiers n'est pas systématique : cela dépend de ce que vous manipulez. Voici les cas où c'est utile, et ceux où ce ne l'est pas.

Dossier du projet — obligatoire

Le Dockerfile doit se trouver à la racine du dossier contenant votre code applicatif, car docker build . prend tout le contenu du dossier courant comme contexte de build.

```
mon-app/
├─ Dockerfile
├─ package.json
├─ server.js
└─ ...
```

Dossier pour les manifestes Kubernetes — recommandé

Rien n'oblige à séparer les fichiers YAML, mais c'est une bonne pratique dès que vous avez plusieurs manifestes (Deployment, Service, ConfigMap...). Cela permet de tout appliquer d'un coup avec un seul dossier en cible.

```
k8s/
├─ deployment.yaml
└─ service.yaml
```

```
kubectl apply -f k8s/
```

Quand ce n'est pas nécessaire

- Pour tester rapidement avec kubectl create deployment ou kubectl expose (commandes impératives) : pas besoin de fichiers ni de dossiers.
- Un seul manifeste YAML peut très bien rester à la racine du projet.

💡 Bon à savoir

Un dossier est indispensable pour le contexte Docker (le code source et le Dockerfile), et optionnel mais conseillé pour organiser les manifestes Kubernetes dès qu'ils se multiplient.

0.3. Faut-il créer un environnement virtuel (venv) ?

Pour une application Python, cela dépend du moment : un venv est utile en développement local, mais inutile — voire déconseillé — une fois dans l'image Docker.

En développement local — recommandé

Un venv isole les dépendances du projet de celles installées globalement sur votre machine, pour éviter les conflits de versions entre projets.

```
python -m venv venv
source venv/bin/activate # Linux / macOS
venv\Scripts\activate    # Windows
pip install -r requirements.txt
```

Ne pas oublier de désactiver l'environnement une fois terminé :

```
deactivate
```

Dans un Dockerfile — inutile

Le conteneur Docker est déjà lui-même un environnement isolé : créer un venv à l'intérieur ajoute une couche inutile qui alourdit l'image sans apporter de bénéfice. Installez les dépendances directement dans le conteneur.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Bon à savoir

Règle simple : venv sur votre machine pour développer et tester, jamais dans l'image Docker. Le fichier requirements.txt (ou pyproject.toml) suffit à reproduire les dépendances dans le conteneur.

Ce point ne concerne que les projets Python — inutile en Node.js (le Dockerfile de l'étape 0 utilise déjà node_modules isolé par nature).

1. Déployer l'application (Deployment)

Un Deployment décrit l'état souhaité de votre application : quelle image utiliser (celle publiée à l'étape 0), combien de réplicas, etc. Kubernetes se charge de créer et maintenir les Pods correspondants.

Étape simple — une seule commande

```
kubectl create deployment mon-app --image=nginx:latest --replicas=2
```

Étape avancée (optionnelle) — via un fichier YAML

Pour bien débiter, la commande ci-dessus suffit. Le fichier YAML ci-dessous fait exactement la même chose, mais de façon réutilisable — utile plus tard, en production.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mon-app
  template:
    metadata:
      labels:
        app: mon-app
    spec:
      containers:
        - name: mon-app
          image: nginx:latest
          ports:
            - containerPort: 80
```

```
kubectl apply -f deployment.yaml
```

Bon à savoir

`kubectl apply` est préférable à `kubectl create` : il est déclaratif. Réappliquer le même fichier met à jour l'existant sans erreur, ce qui facilite le versionnement (GitOps).

2. Vérifier le déploiement (Pods)

Étape simple — voir si ça tourne

```
kubectl get deployments
kubectl get pods
```

Si la colonne STATUS affiche Running, tout va bien.

Étape avancée (optionnelle) — diagnostiquer un problème

Si un Pod ne démarre pas, ces commandes donnent plus de détails.

```
kubectl describe pod <nom-du-pod>
```

```
kubectl logs -f <nom-du-pod>
```

Statuts à connaître

- Pending : le Pod attend d'être planifié sur un node (souvent un problème de ressources).
- ContainerCreating : l'image est en cours de téléchargement.
- Running : tout va bien.
- CrashLoopBackOff : le container plante en boucle — vérifier les logs.

3. Exposer l'application (Service)

Un Service donne une adresse réseau stable à vos Pods (dont les IP changent à chaque redémarrage) et répartit la charge entre eux.

Étape simple — une seule commande

```
kubectl expose deployment mon-app --port=80 --target-port=80 --type=NodePort
```

```
kubectl get services
```

Étape avancée (optionnelle) — choisir le bon type de Service

- ClusterIP (par défaut) : accessible uniquement à l'intérieur du cluster.
- NodePort : expose l'app sur un port fixe de chaque node, accessible depuis l'extérieur — c'est celui utilisé ci-dessus, pratique pour tester en local.
- LoadBalancer : provisionne un load balancer externe (AWS, GCP, Azure...), pour la production.

4. Tester la connectivité

Étape simple — avec port-forward

La méthode la plus simple pour tester rapidement, quel que soit le type de Service.

```
kubectl port-forward service/mon-app 8080:80
```

```
curl http://localhost:8080
```

Étape avancée (optionnelle) — autres méthodes

Depuis l'intérieur du cluster, avec un Pod temporaire :

```
kubectl run test-curl --image=curlimages/curl -it --rm -- curl http://mon-app
```

Si le Service est en NodePort, directement via l'IP du node :

```
kubectl get service mon-app
curl http://<IP-du-node>:<NodePort>
```

5. Scaler si nécessaire

Étape simple — scaling manuel

```
kubectl scale deployment mon-app --replicas=5
```

```
kubectl get pods -w
```

Le drapeau -w (watch) affiche les changements en temps réel.

Étape avancée (optionnelle) — scaling automatique

Nécessite metrics-server installé sur le cluster. Kubernetes ajuste alors le nombre de réplicas tout seul selon la charge CPU.

```
kubectl autoscale deployment mon-app --min=2 --max=10 --cpu-percent=80
```

```
kubectl get hpa
```

+ . Commande bonus : tout voir d'un coup

```
kubectl get all -l app=mon-app
```

Affiche le Deployment, les Pods, le Service et éventuellement le HPA associés au label app=mon-app.